

2-2

객체 지향 언어

객체는 자체의 속성과 행동을 함께 가지는 독립적인 단위로, 현실 세계의 사물이나 개념을 소프트웨어적으로 모델링한 것이다.

간단하게 설명하면 자체적인 변수와 함수를 갖는 새로운 속성의 코드 정도로 이해해도 좋다.

등장 배경

소프트웨어가 점점 더 복잡해지고 크기가 커지면서 기존 절차지향언어로는 코드 관리와 유지보수가 어려워 졌다. 개발자들은 코드 재사용 및 모듈화를 통해 빠르고 정확한 개발을 원했다. 객체 지향 언어는 프로그램을 하나의 '흐름'이란 개념이 아닌 다수의 '객체'로 구성한다.

예를 들어, 건물의 위치와 기능, 관계도 등의 정보를 저장하여 지도로 시각화하는 프로그램을 개발한다고 가정할 때 만약 절차지향언어를 사용해서 하나의 흐름으로 프로그램을 개발한다면 코드가 너무 복잡해지고 새로운 건물이 생겨나는 등 프로그램을 수정할 때 매우 어려울 것이다.

객체지향언어라면 각 건물을 하나의 객체로 구현하고 다수의 객체를 동시에 원하는 만큼 사용하는 것으로 이를 구현할 수 있을 것이다. 만약 다른 지역의 동일한 프로그램을 사용하는 경우에도 해당 프로그램을 조금만 수정해서 구현할 수 있을것이다.

함수

함수는 입력값(x)에 따라 출력값(y)이 변하는 것, 이것을 함수라고 한다.

예를 들어 우리가 믹서기에 과일을 넣으면 주스를 만들수 있다. 믹서에 넣는 과일은 '입력', 과일 주스는 '출력(결과값)'이 된다.

```
def add(a, b):  
    result = a + b  
    return result
```

PYTHON

함수를 사용하는 이유

프로그래밍을 하다 보면 같은 내용을 반복해서 작성하는 경우가 자주 있다. 이럴 때 같은 내용을 반복해서 코딩을 하지 않고, 반복적으로 사용되는 가치 있는 부분을 한 뭉치로 묶어 어떤 입력값을 주었을 때

어떤 결과값을 반환해 주는 함수를 작성한다.

이렇게 특정 코드를 묶어 하나의 함수로 사용하면 계속 재사용할 수 있기 때문에 반복적인 프로그래밍을 피할 수 있다. 또한 이렇게 함수를 통해 묶어서 사용하면 유지 보수 및 코드 가독성측면에서도 유리하다.

PYTHON

```
# 함수 정의
def greet(name):
    print(f"안녕하세요, {name}님!")
```

```
# 함수 호출
greet("민수")
greet("지영")
```

```
>>>안녕하세요, 민수님!
>>>안녕하세요, 지영님!
```

예: 함수를 사용하지 않을 경우

```
# 학생 1 성적 처리
name1 = "민수"
kor1 = 90
eng1 = 85
math1 = 92

total1 = kor1 + eng1 + math1
avg1 = total1 / 3

print("이름:", name1)
print("총점:", total1)
print("평균:", avg1)

if avg1 >= 90:
    print("등급: A")
elif avg1 >= 80:
    print("등급: B")
elif avg1 >= 70:
    print("등급: C")
else:
    print("등급: D")

print("-----")

# 학생 2 성적 처리
name2 = "지영"
kor2 = 78
eng2 = 88
math2 = 84

total2 = kor2 + eng2 + math2
avg2 = total2 / 3

print("이름:", name2)
print("총점:", total2)
print("평균:", avg2)

if avg2 >= 90:
    print("등급: A")
elif avg2 >= 80:
    print("등급: B")
elif avg2 >= 70:
    print("등급: C")
else:
    print("등급: D")

print("-----")
```

```
# 학생 3 성적 처리
name3 = "현우"
kor3 = 65
eng3 = 70
math3 = 72

total3 = kor3 + eng3 + math3
avg3 = total3 / 3

print("이름:", name3)
print("총점:", total3)
print("평균:", avg3)

if avg3 >= 90:
    print("등급: A")
elif avg3 >= 80:
    print("등급: B")
elif avg3 >= 70:
    print("등급: C")
else:
    print("등급: D")

print("-----")
```

함수를 사용 할 경우

```
def print_student_result(name, kor, eng, math):
    total = kor + eng + math
    avg = total / 3

    print("이름:", name)
    print("총점:", total)
    print("평균:", avg)

    if avg >= 90:
        print("등급: A")
    elif avg >= 80:
        print("등급: B")
    elif avg >= 70:
        print("등급: C")
    else:
        print("등급: D")

    print("-----")

print_student_result("민수", 90, 85, 92)
print_student_result("지영", 78, 88, 84)
print_student_result("현우", 65, 70, 72)
```

출력 결과

```
이름: 민수
총점: 267
평균: 89.0
등급: B
-----
이름: 지영
총점: 250
평균: 83.33333333333333
등급: B
-----
이름: 현우
총점: 207
평균: 69.0
등급: D
-----
```

함수를 사용하면 프로그램을 기능 단위로 함수로 분리해 놓아 프로그램의 흐름을 일목 요연하게 볼 수 있다.

파이썬 함수의 구조

함수의 구조 정의

파이썬 함수의 구조는 아래와 같다.

```
def 함수_이름(매개변수):  
    수행할_문장1  
    수행할_문장2  
    ...
```

PYTHON

- def는 함수를 만들때 사용하는 예약어이다. 함수 이름은 만드는 사람이 임의로 만드는 게 가능하며, 함수 이름 뒤 괄호 안에 매개 변수는 입력으로 전달 되는 값을 받는 변수다.
- 함수를 정의한 후 if, while, for 문 등과 마찬가지로 함수에서 문장을 입력한다.

```
def add(a, b):  
    return a + b
```

PYTHON

위 함수는 이렇게 정의할 수 있다.

"이 함수의 이름은 **add**이고 입력으로 **2**개의 값을 받으며 반환값(출력값)은 **2**개의 입력값을 더한 값이다."

함수 호출하기

```
def add(a, b):  
    return a + b  
  
result = add(5, 3) # 함수를 호출  
print(result)
```

PYTHON

- 해당 코드를 실행하면 결과값으로 8이 출력된다.
- 함수를 호출할 때는 사용할 함수의 이름을 명시한 뒤, 필요한 인자값을 전달해 주면 된다.

매개변수와 반환값

```
def multiply(x, y):  
    result = x * y  
    return result  
  
product = multiply(4, 6)  
print(product)
```

PYTHON

- 함수는 매개변수(인자값)를 받아들일 수 있고, 블록안에 정의된 로직을 수행한 뒤 값을 반환할 수 있다. 매개변수는 함수 내에서 사용되는 변수이며, 반환값은 함수 호출 결과로 반환된다.
- 함수 내에서 return 키워드를 사용하여 값을 반환한다. 이런 반환 매개변수와 반환 값은 선택사항이며 경우에 따라 사용하지 않아도 된다.

입력값이 없는 함수

PYTHON

```
def say():  
    return 'Hi'  
  
a = say()  
print(a)
```

a

- 입력값없이 함수가 존재할 수 있으며, `say()` 처럼 괄호 안에 아무런 값도 넣지 않아야 한다. 이 함수는 입력값은 없지만, 반환값으로 "HI"라는 문자열을 반환한다.

함수 예제

PYTHON

```
# 1. 전달받은 인자값을 출력하는 함수
def setValue(newValue):
    x = newValue
    print("인자값:", x)

# 함수 호출
setValue(5)

#=====#
# 2. 전달받은 2개의 인자값의 순서를 바꿔서 출력하는 함수
def swap(x,y):
    return y,x

# 함수 호출
print(swap(3,4))

#=====#
# 3. 교집합 문자를 리턴하는 함수
def intersect(prelist, postlist):
    #지역변수
    result = []
    #H(0) | A(1) | M(2)
    for x in prelist:
        #x라는 글자가 postlist에 있고 x가 result에 없으면
        if x in postlist and x not in result:
            result.append(x)
    return result

#함수 호출
print(intersect("HAM", "SPAM"))
```

PYTHON

```
#키워드 인자
def connectURI(server, port):
    strURL = "https://" + server + ":" + port
    return strURL

#함수 호출
print( connectURI(port="8080", server="naver.com"))
```

입력값이 여러가지 인 경우

입력값이 여러 개일 때 그 입력값을 모두 더해 주는 함수를 생각해 보자. 하지만 몇 개가 입력될 지 모를 때는 어떻게 해야 할까? 파이썬은 다음과 같은 방법이 존재한다.


```
def 함수_이름(*매개변수):
    수행할_문장
```

- 일반적으로 볼 수 있는 함수 형태에서 괄호 안의 매개변수 부분이 ***매개 변수** 바뀌었다.

```
def add_many(*args):
    result = 0
    for i in args:
        result = result + i
    return result
```

- add_many 함수는 입력값이 몇 개이든 상관없이 매개변수 이름 앞에 *****을 붙이면 입력값을 전부 모아 튜플로 만들어 주기 때문이다.
- 만약 **add_many(1, 2, 3)**처럼 쓰면 args는 (1, 2, 3)이 되고 **add_many(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)**이 된다.
- 여러 개의 입력을 처리 할때 **def add_many(*args)**처럼 함수의 매개변수로 ***args** 하나만 사용할 수 있는 것은 아니다.

```
def add_mul(choice, *args):
    if choice == "add":
        result = 0
    else:
        result = 1

    return result

result = add_mul('add', 1, 2, 3, 4, 5)
print(result)
```

클래스(class)

파이썬에서 클래스는 서로 관련 있는 데이터와 기능을 하나로 묶어서 표현하는 설계도다.

프로그램에서 어떤 대상을 표현하려면 그 대상이 가지는 데이터(속성)과 기능(동작)이 필요함. 이때 어떤 데이터가 있는지 어떤 기능을 하는지를 미리 정의해 놓은 것을 **클래스**라고 한다. 이 클래스를 이용해 실제로 만들어진 것을 **객체(Object)**라고 한다.

- 특징: 속성과 기능을 함께 묶을 수 있다. 그래서 데이터와 그 데이터를 다루는 동작을 한 곳에서 관리할 수 있다. 하나의 클래스를 만들어 두면 그 구조를 바탕으로 여러 개의 객체를 만들 수 있다. 예시) 자동차 설계도가 있으면 그 설계도를 바탕으로 다른 자동차도 양산이 가능

- 장점: 코드를 체계적으로 관리가 가능해짐. 관련 있는 내용을 묶어 놓기 때문에 코드가 길어져도 구조를 파악하기 쉬워진다.

1. 클래스 정의

구성 요소:

- 필드(Field): 클래스 내부에 선언된 변수로, 객체의 상태 데이터를 저장한다.
- 메서드(Method): 클래스 내부에서 구현된 함수를 의미하며, 객체의 기능을 정의한다.
- 자동차 클래스의 개념을 실제 코드로 구현

SHELL

```
class Car :  
    # 자동차의 필드  
    색상 = ""  
    현재_속도 = 0  
  
    자동차의 메서드  
    def upSpeed(증가할_속도량) :  
        # 현재 속도에서 증가할_속도량만큼 속도를 올리는 코드  
  
    def downSpeed(감소할_속도량) :  
        # 현재 속도에서 감소할_속도량만큼 속도를 내리는 코드
```

- 클래스는 class 키워드를 이용하여 생성한다.
- 먼저 클래스의 이름을 정하고 그 클래스가 가질 속성과 기능을 설계한다. 즉, 어떤 데이터를 저장하고 어떤 동작을 수행할지 미리 정해서 클래스라는 틀을 만드는 단계이다.

PYTHON

```
class Car:  
    # 자동차의 필드  
    색상 = ""  
    현재_속도 = 0
```

2. 함수가 있는 클래스

- 클래스 안에는 변수 뿐만 아니라 함수도 함께 정의할 수 있다. 클래스안에 들어 있는 함수를 메서드라고 하며, 객체가 수행할 수 있는 동작이나 기능을 나타낸다.

```
class Car :
    # 자동차의 필드
    Color = ""
    Speed = 0

    # 자동차의 메서드
    def Up_Speed(증가할_속도량) :
        print("속도를 올립니다.")
    def Down_Speed(감소할 속도량) :
        print("속도를 내립니다.")
```

구현 형식 :

- class 클래스명: 키워드를 사용하여 관련 코드를 블록 내에 구현한다.

인스턴스(Instance)와 self 키워드

- 인스턴스 생성: 선언된 클래스를 바탕으로 메모리에 실제 객체를 구현하는 과정. 하나의 클래스로부터 복수의 독립적인 인스턴스를 생성할 수 있다.
- self의 역할: 메서드 정의 시 첫번째 매개변수로 반드시 기술해야 하며, 이는 메서드를 호출한 인스턴스 자신을 가리킨다..
- self 생략: 메서드 내부에서 클래스의 필드를 참조하거나 사용하지 않을 경우에 한해 생략이 가능하다.

```
class Car :
    color = ""
    speed = 0

    def upSpeed(sef, value) :
        self.speed +=value

    def downSpeed(self, value) :
        self.speed -= value
```

메서드 호출(Method Call)

메모리에 할당된 인스턴스를 식별하여 클래스 내부에 정의된 특정 로직을 가동하는 과정

1. 호출 형식: 인스턴스명,.메서드(입력값) 의 구조로 실행

```
myCar1.upSpeed(30)
myCar2.downSpeed(60)
```

클래스의 완전한 작동 구현

클래스 사용 순서

단계	작업	형식	예
1단계	클래스 선언	class 클래스명: # 필드 선언 # 메서드 선언	class Car: color = " " def.upSpeed(self.value)"
2단계	인스턴스 생성	인스턴스 = 클래스명()	myCar1 = Car()
3단계	필드나 메서드 사용	인스턴스.필드명 = 값 인스턴스.메서드()	myCar1.color = "빨강" myCar1.upSpeed(30)

- 사용 예시

```

class Car :# car 클래스 정의
    color = "" # 자동차의 색상
    speed = 0 # 자동차의 속도

    def upSpeed(self, value) : #매개 변수를 추가 속도(value)를 받아 현재 속도 증가
        self.speed +=value

    def downSpeed(self, value) : #매개 변수를 추가 속도(value)를 받아 현재 속도 감소
        self.speed -= value

myCar1 = Car()
myCar1.color = "빨강"
myCar1.speed = 0

myCar2 = Car()
myCar2.color = "파랑"
myCar2.speed = 0

myCar3 = Car()
myCar3.color = "노랑"
myCar3.speed = 0

myCar1.upSpeed(30)
print("자동차1의 색상은 %s이며, 현재 속도는 %dkm입니다." % (myCar1.color,
myCar1.speed))

myCar2.upSpeed(60)
print("자동차2의 색상은 %s이며, 현재 속도는 %dkm입니다." % (myCar2.color,
myCar2.speed))

myCar3.upSpeed(0)
print("자동차3의 색상은 %s이며, 현재 속도는 %dkm입니다." % (myCar3.color,
myCar3.speed))

```

자동차1의 색상은 빨강이며, 현재 속도는 30km입니다.
 자동차2의 색상은 파랑이며, 현재 속도는 60km입니다.
 자동차3의 색상은 노랑이며, 현재 속도는 0km입니다.

생성자(Constructor)의 개념

인스턴스가 생성되는 시점에 호출되어 필드값을 초기화하는 특수 함수

1. 자동 실행: **init()** 는 인스턴스가 생성되는 시점에 시스템에 의해 자동으로 호출되는 특수 메서드다.

- 언더바가 2개 붙은 것은 파이썬에서 예약해 놓은 것, 프로그램을 작성시 이름을 사용해서 새로운 함수나 변수명을 만들지 말 것
- 인스턴스 생성 시 전달된 인자값을 매개변수로 받아 필드에 할당함으로써 객체의 초기 상태를 설정

```
class 클래스명 :
    def __init__(self) :
        # 이부분에 초기화할 코드 입력
```

SHELL

```
class Car :
    color = ""
    speed = 0

    def __init__(self) :
        self.color = "빨강"
        self.speed = 0
```

2. 유형:

- 기본 생성자: 매개변수가 self만 존재하며 내부적으로 고정된 값으로 초기화

SHELL

```
class Car:
    color = ""
    speed = 0

    def __init__(self):
        self.color = "빨강"
        self.speed = 0

    def upSpeed(self, value):
        self.speed += value

    def downSpeed(self, value):
        self.speed -= value

myCar1 = Car()
myCar2 = Car()

print("자동차1의 색상은 %s이며, 현재 속도는 %dkm이다." % (myCar1.color, myCar1.speed))
print("자동차2의 색상은 %s이며, 현재 속도는 %dkm이다." % (myCar2.color, myCar2.speed))
```

```
>>> 자동차1의 색상은 빨강이며, 현재 속도는 0km이다.
>>> 자동차2의 색상은 빨강이며, 현재 속도는 0km이다.
```

- 매개변수가 있는 생성자: 인스턴스 생성 시 인자를 전달받아 실행 시점의 입력값에 따라 필드를 초기화한다.

SHELL

```
class Car:
    color = ""
    speed = 0

    def __init__(self, value1, value2):
        self.color = value1
        self.speed = value2

myCar1 = Car("빨강", 30)
myCar2 = Car("파랑", 60)

print("자동차1의 색상은 %s이며, 현재 속도는 %dkm이다." % (myCar1.color, myCar1.speed))
print("자동차2의 색상은 %s이며, 현재 속도는 %dkm이다." % (myCar2.color, myCar2.speed))
```

```
>>> 자동차1의 색상은 빨강이며, 현재 속도는 30km이다.
>>> 자동차2의 색상은 파랑이며, 현재 속도는 60km이다.
```

인스턴스 변수와 클래스 변수

클래스 변수

1. 인스턴스 변수: 각 인스턴스(객체)마다 개별적으로 할당되는 변수다. 각 객체는 자신만의 독립적인 메모리 공간을 가지며 서로 데이터를 공유하지 않는다.

SHELL

```
class Car :
    color = ""
    speed = 0
```

2. 클래스 변수 : 클래스 내부에 정의되어 해당 해당 클래스로 생성된 모든 인스턴스가 공동으로 사용하는 변수다. 메모리 공간이 클래스 차원에서 하나만 할당되므로 모든 객체가 값을 공유

```
myCar1 = Car()
myCar2 = Car()
```

3. 클래스 변수 활용

```
class Car:
    color = ""
    speed = 0
    count = 0 # 클래스 변수

    def __init__(self):
        self.speed = 0
        Car.count += 1

# 객체 생성
myCar1 = Car()
myCar1.speed = 30
print("자동차1의 속도는 %dkm이며, 현재 자동차 수는 %d대이다." % (myCar1.speed,
Car.count))

myCar2 = Car()
myCar2.speed = 60
print("자동차2의 속도는 %dkm이며, 현재 자동차 수는 %d대이다." % (myCar2.speed,
Car.count))
```

```
>>> 자동차1의 속도는 30km이며, 현재 자동차 수는 1대이다.
>>> 자동차2의 속도는 60km이며, 현재 자동차 수는 2대이다.
```

클래스는 왜 필요한가?

파이썬으로 잘 만든 프로그램을 살펴봐도 클래스를 사용하지 않고 작성한 것이 매우 많다. 즉 클래스는 지금까지 공부한 함수나 자료형처럼 프로그램 작성을 위해 꼭 필요한 요소는 아니다.


```
# calculator3.py
class Calculator:
    def __init__(self):
        self.result = 0

    def add(self, num):
        self.result += num
        return self.result

cal1 = Calculator()
cal2 = Calculator()

print(cal1.add(3))
print(cal1.add(4))
print(cal2.add(3))
print(cal2.add(7))
```

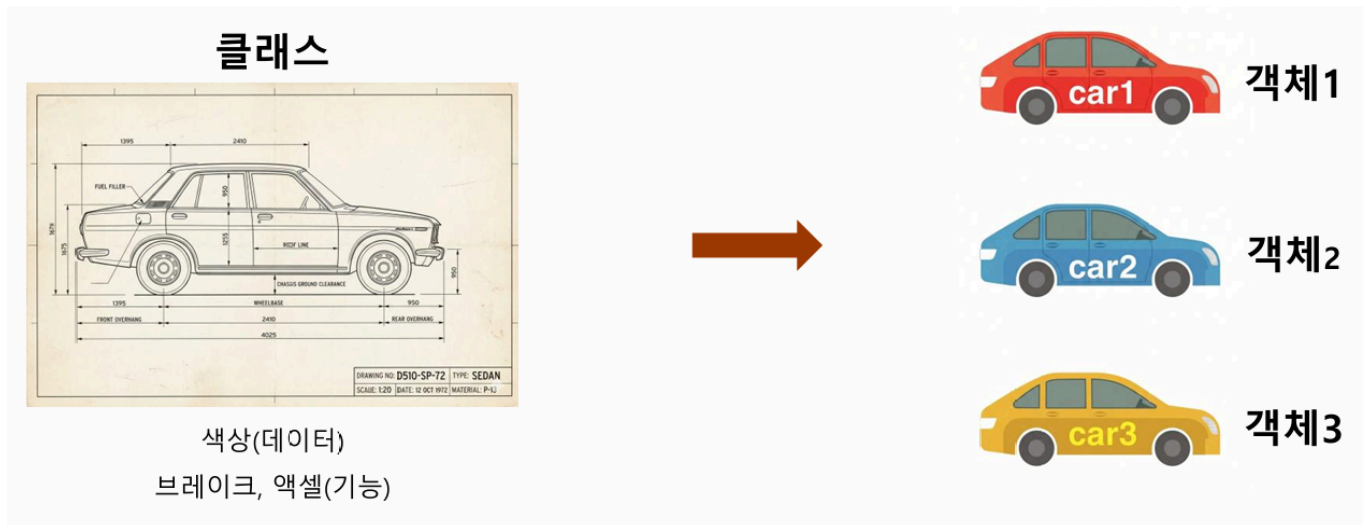
- Calculator 클래스로 만든 별개의 계산기 cal1, cal2(파이썬에서는 이것을 '객체'라고 부른다.)가 각각의 역할을 수행한다. 그리고 계산기의 결괏값 역시 다른 계산기의 결괏값과 상관없이 독립적인 값을 유지한다.
- 빼기 기능을 더하고 싶다면 `Calculator` 클래스에 다음과 같이 빼기 기능을 가진 함수를 추가하면 된다.

```
class Calculator:
    def __init__(self):
        self.result = 0

    def add(self, num):
        self.result += num
        return self.result

    def sub(self, num):
        self.result -= num
        return self.result
```

클래스와 객체



- 클래스(class)란 똑같은 무언가를 계속 만들어 낼 수 있는 설계 도면(과자 틀)이고, 객체(object)란 클래스로 만든 피조물(과자 틀로 찍어 자동차)을 뜻한다.
- 클래스로 만든 객체의 중요한 특징은 바로 객체마다 고유한 성격을 가진다는 것이다. 동일한 클래스로 만든 객체들은 서로 전혀 영향을 주지 않는다.

클래스 정의

- 클래스는 `class` 키워드를 이용하여 생성한다. 클래스 블록 내부에 변수와 함수를 작성할 수 있다.
- 클래스 내부에 선언된 함수를 '메서드'라고 한다. 아래와 같이 일반적으로 작성하는 메서드를 '인스턴스 메서드'라고 하며, 매개 변수로 반드시 `self`를 넣어야 한다. 이렇게 정의한 클래스를 이용해 객체들을 생성할 수 있다.
- 객체나 메서드나 변수를 사용하기 위해서는 `'!'`을 사용하여 해당 변수나 메서드를 불러와야 한다.

PYTHON

```
class Car():
    Color = "" #내부 변수(데이터)
    Speed = 0

    def speedUp(self):
        print("속도를 올립니다.") # 메서드(기능)
    def speedDown(self):
        print("속도를 내립니다.")

car1 = Car():
car2 = Car():
car3 = Car():

car1.Color = "Red"
car2.Speed = 10
car3.speedUp()
```

self

- self는 인스턴스 메서드 안에서 객체 자기 자신을 가리킨다.
- 객체의 변수에 접근할 때는 self.변수명, 객체의 다른 메서드를 호출할 때는 self.메서드명()을 사용한다.
- 같은 클래스로 여러 객체를 만들더라도 각 객체의 self는 서로 다른 자기 자신을 가리키므로, 각 객체는 인스턴스 변수를 독립적으로 가진다.

특수 메서드 init

- 메서드 중에서 앞 뒤로 `__`가 두 개씩 붙은 메서드를 특수 메서드라고 하며, 특정 상황에 자동으로 호출된다.
- `__init__`은 초기화 메서드라고 하며, 객체 생성 시 자동으로 호출되는 기능이 있다.
- 초기화 메서드는 객체 생성 시 인스턴스 변수들을 원하는 값으로 설정하거나 처음 필요한 작업을 수행할 때 사용한다.
- 초기화 메서드의 매개변수는 객체 생성 시 ()안에 넣어 사용할 수 있다.

PYTHON

```
class Car :
    def __init__(self, color, speed):
        self.Color = color # 객체 생성 시 매개변수로 입력한 color를 Color 인스턴스 변수
        #에 대입
        self.Speed = speed # 객체 생성 시 매개변수로 입력한 speed를 Speed 인스턴스 변수
        #에 대입

    def set_color(self, color):
        self.Color = color # 객체 내부의 Color 인스턴스 변수 참조

    def print_speed(self):
        print(f"현재 속도는 {self.Speed}입니다.") # 객체 내부의 Speed 인스턴스 변수 참조

    def speedUp(self, speed):
        self.Speed += speed # 객체 내부의 Color 인스턴스 변수 참조
        self.print_speed() # 객체 내부의 print_speed() 메서드 참조

    def speedDown(self, speed):
        self.Speed -= speed
        self.print_speed()

car = Car("White", 50)
print(car.Color) # White 출력
print(car.Speed) # 50 출력
```

모듈

- Python에서도 외부 파일에 작성된 코드를 가져와서 사용할 수 있다.
- 이때 .py 파일 하나를 '모듈' 이라고 하며, 여러 모듈들을 모아 둔 폴더를 '패키지' 라고 한다.
- 여러 모듈과 패키지들을 모아 둔 조합을 '라이브러리'라고 한다.
- Python에서는 import문을 사용하여 모듈을 사용할 수 있다. 모듈 안의 변수, 함수, 클래스는 '.'리호를 붙여서 사용할 수 있다.
- 모듈 검색 경로 중 하나는 실행 파일과 같은 위치다.

PYTHON

```
# my_module.py
a = 10

def print_5():
    print(5)

class module_class():
    a = 7
```

PYTHON

```
# main.py

from my_module import a, print_5, module_class

print(a)
print_5()
module_class_1 = module_class()
print(module_class_1.a) #7출력
```

Package

패키지는 폴더 안에 모듈(.py)들이 들어있는 형태를 패키지라고 한다. 패키지의 모듈들을 가져올 때도 from ~import 문이나 '.' 기호를 사용하여 가져올 수 있다.

폴더1, 폴더 2처럼 코드를 작성하여 모듈을 가져올 수 있으며, from에 모듈을 덧붙여 원하는 모듈 내 변수, 함수, 클래스만 가져오는것도 가능하다.

PYTHON

```
# main.py
from package1 import module1 # module1.py를 import 하는 방법 1
import package1.module1      # module1.py를 import 하는 방법 2

from package1.package2 import module2 # module2.py를 import 하는 방법 1
import package1.package2.module2      # module2.py를 import 하는 방법 2

from package1.package2.module2 import a # module2.py 안의 a를 import 하는 방법
```

import문의 특수한 기능들

`import ~as` 문을 사용하면 모듈을 내가 원하는 명칭으로 사용할 수 있다.
`from [패키지] import *` 문을 사용하면 패키지 폴더 내의 모든 모듈을 import 할 수 있다.

내장 라이브러리(Standard Library)

- Python에 기본으로 포함되어 있는 라이브러리를 말한다.
- 사용자가 따로 설치하지 않아도 바로 모듈을 import 하여 편리하게 사용할 수 있다.
- `dir('모듈')` 명령어로도 제공하는 모듈과 예약어를 확인할 수 있다.

PYTHON

```
import sys
print(dir(sys))
```

내장 모듈	주 용도	간단한 예
<code>math</code>	수학 함수	<code>math.sqrt(16)</code>
<code>random</code>	난수 생성	<code>random.randint(1, 10)</code>
<code>datetime</code>	날짜와 시간 처리	<code>datetime.datetime.now()</code>
<code>time</code>	시간 측정, 지연	<code>time.sleep(1)</code>
<code>os</code>	운영체제 기능	<code>os.getcwd()</code>
<code>sys</code>	파이썬 실행 환경 제어	<code>sys.argv</code>
<code>pathlib</code>	파일 경로 처리	<code>Path("data.txt")</code>
<code>json</code>	JSON 데이터 처리	<code>json.loads(text)</code>
<code>csv</code>	CSV 파일 읽기/쓰기	<code>csv.reader(file)</code>
<code>re</code>	정규표현식	<code>re.findall(pattern, text)</code>
<code>collections</code>	특수 자료구조	<code>Counter(["a", "a", "b"])</code>
<code>itertools</code>	반복자 도구	<code>itertools.permutations([1, 2])</code>
<code>functools</code>	함수형 프로그래밍 도구	<code>functools.reduce()</code>
<code>copy</code>	객체 복사	<code>copy.deepcopy(obj)</code>
<code>pickle</code>	파이썬 객체 저장	<code>pickle.dump(obj, file)</code>
<code>sqlite3</code>	SQLite 데이터베이스	<code>sqlite3.connect("test.db")</code>
<code>urllib</code>	URL 요청 처리	<code>urllib.request.urlopen(url)</code>
<code>threading</code>	스레드 처리	<code>threading.Thread()</code>
<code>multiprocessing</code>	멀티프로세싱	<code>multiprocessing.Process()</code>
<code>subprocess</code>	외부 프로그램 실행	<code>subprocess.run(["ls"])</code>

내장 모듈	주 용도	간단한 예
<code>typing</code>	타입 힌트	<code>list[int]</code>
<code>enum</code>	열거형 정의	<code>Enum</code>
<code>dataclasses</code>	데이터 클래스 생성	<code>@dataclass</code>
<code>argparse</code>	명령행 인자 처리	<code>argparse.ArgumentParser()</code>
<code>logging</code>	로그 기록	<code>logging.info("message")</code>

math 모듈

복잡한 계산에 사용되는 모듈이다.
제곱근, 거듭제곱, 삼각함수, 원주율 같은 복잡한 연산 기능이 필요할 때 사용한다.
기본 연산자로 구현되지 않는 복잡한 연산을 내부의 함수를 통해 지원한다.

PYTHON

```
import math

print(math.sqrt(16))
print(math.pow(2, 3))
print(math.pi)
```

함수	설명
<code>math.sqrt(x)</code>	제곱근
<code>math.pow(a, b)</code>	a의 b 제곱
<code>math.ceil(x)</code>	올림
<code>math.floor</code>	내림
<code>math.pi</code>	원주율

random 모듈

무작위 값을 만들어 주는 모듈이다.
임의 값을 뽑거나 데이터를 섞을 때 사용한다.
사용자는 범위를 지정하거나 무작위 값을 뽑거나, 복합 자료형의 요소들을 섞을 수 있다.

```
import random

print(random.randint(1, 10))
print(random.random())

fruits = ["apple", "banana", "grape"]
print(random.choice(fruits))
```

함수	설명
random.randint(a, b)	a부터 b까지 정수 중 하나를 랜덤으로 선택
random.random()	0 이상 1미만의 실수 생성
random.choice(리스트)	리스트에서 하나 선택
random.shuffle(리스트)	리스트 순서를 섞음

time 모듈

시간과 관련된 기능을 제공하는 모듈이다.
프로그램 실행 시간 확인, 일정 시간 정지, 현재 시간을 가져올 때 사용한다.

```
import time

print(time.time())    # 현재 시간(초 단위)
time.sleep(2)         # 2초 동안 멈춤
print("2초 후 실행")

now = time.localtime()
print(time.strftime("%Y-%m-%d %H:%M:%S", now))
```

실전

1. 함수로 두 수 더하기

```
def add(a, b):  
    result = a + b  
    return result  
  
num1 = 10  
num2 = 20  
  
answer = add(num1, num2)  
print("결과:", answer)
```

2. 여러 개의 점수를 받아 평균 구하기

```
def get_average(*scores):  
    total = 0  
  
    for score in scores:  
        total += score  
  
    average = total / len(scores)  
    return average  
  
result = get_average(90, 80, 70, 100)  
  
print("평균:", result)
```

3. 클래스로 계산기 만들기


```
class Calculator:
    def __init__(self):
        self.result = 0

    def add(self, num):
        self.result += num
        return self.result

    def sub(self, num):
        self.result -= num
        return self.result

cal = Calculator()

print(cal.add(10))
print(cal.add(5))
print(cal.sub(3))
```

4. time 내장 라이브러리 사용하기

```
import time

print("프로그램 시작")

time.sleep(2)

print("2초 후 실행")

now = time.localtime()
print(time.strftime("%Y-%m-%d %H:%M:%S", now))
```

문제

문제 1. 두 수의 합과 곱을 구하는 함수 만들기

조건

- 함수 이름은 `calculate`
- 매개변수는 `a`, `b`
- 출력 형식은 아래와 같게 하기

실행 예시

```
calculate(3, 5)
```

출력 예시

```
합 : 8  
곱 : 15
```

문제 2. 학생 평균과 등급 출력하기

학생 이름과 국어, 영어, 수학 점수를 입력받아 총점, 평균, 등급을 출력하는 함수를 작성하시오.

조건

- 함수 이름은 `print_student_result`
- 평균이 90 이상이면 A
- 평균이 80 이상이면 B
- 평균이 70 이상이면 C
- 그 외는 D

실행 예시

```
print_student_result("민수", 90, 85, 92)
```

*출력 예시

```
이름 : 민수  
총점 : 267  
평균 : 89.0  
등급 : B
```